

# BLM101 – Bilgisayar Mühendisliğine Giriş Dersi Dönem Projesi

*Pınar Nida TUNCA*

*25360859206*

# İÇERİK:

1. Metin, resim ve ses verilerinin bit düzeyinde temsili
2. Veri sıkıştırma neden gereklidir?
3. Run-Length Encoding (RLE) gibi temel sıkıştırma mantıkları

# 1.BÖLÜM:

## Her Şey Birler ve Sıfırlardan İbaret

Bilgisayarlar dünyayı bizim gördüğümüz gibi görmez. Metinler, fotoğraflar, sesler ve aklınıza gelebilecek her türlü bilgi, en temel düzeyde ‘bit’ adı verilen ikili rakamların (0 veya 1) dizileri olarak temsil edilir. Buna ‘bilginin sayısallaştırılması’ denir.



### Metin

Her harf, rakam ve sembol benzersiz bir bit deseniyle eşleştirilir.

01000001



### Görüntü

Resimler, her biri bir renk değerini temsil eden bit desenlerinden oluşan küçük noktaların (piksellerin) bir mozağıdır.

11110000...

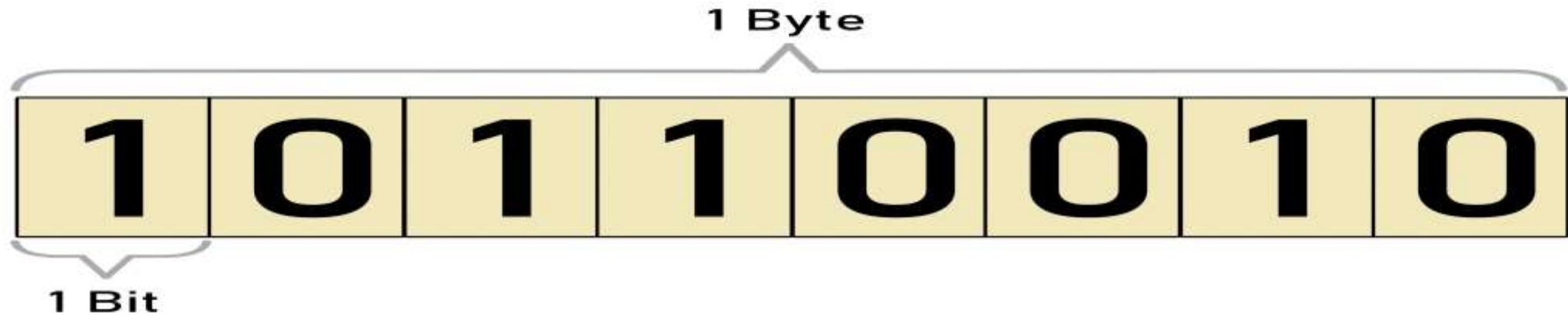


### Ses

Ses dalgaları, belirli zaman aralıklarında genlikleri ölçülerek (örnekleme) bir dizi sayıya ve dolayısıyla bit desenine dönüştürülür.

01011010...

# Bit and Byte



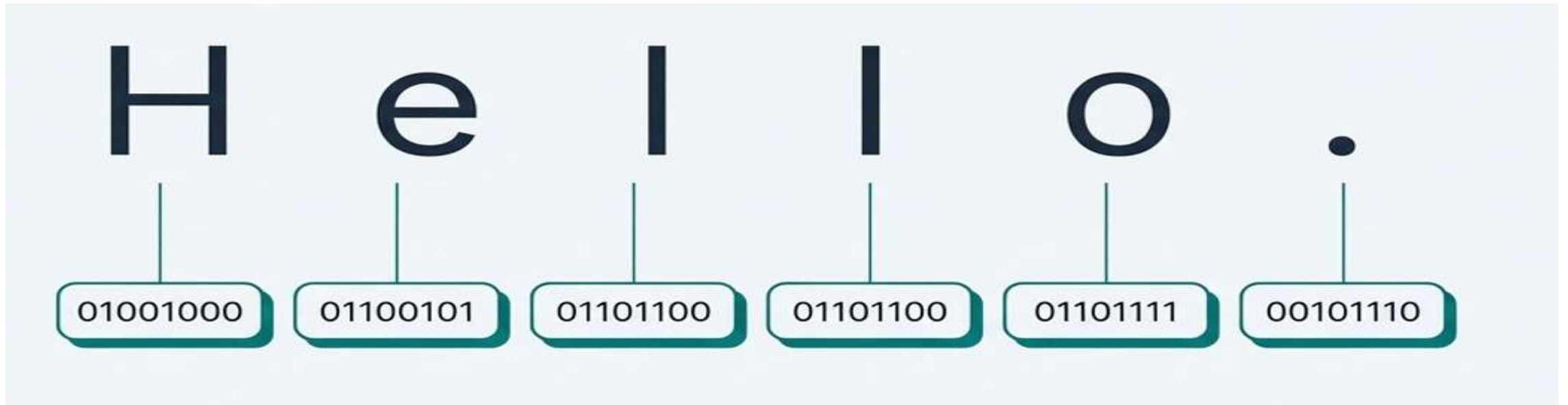
- **Bit**, bilgisayarda bilginin en küçük birimidir. Adı, İngilizce **Binary Digit** (İkili Rakam) kelimelerinin kısaltmasıdır.
- **Fiziksel Karşılık:** Bir transistörün durumunu (açık/kapalı), bir voltaj seviyesini (yüksek/düşük) veya bir manyetik alanın yönünü temsil eder.
- 8 adet bit bir araya gelerek **1 Bayt(Byte)** oluşturur. Bir bayt, 256 farklı değeri veya tek bir karakteri (harf, rakam) temsil edebilir.
- **Unutma:** Bilgisayarların "anladığı" tek dil ikili dildir (0 ve 1). Tüm karmaşık veriler, bu ikili sistem kullanılarak kodlanır.

# Hexadecimal (Onaltılık) Gösterim

- **Hexadecimal (16'lık)** sistem, 16 farklı rakam veya sembol kullanarak sayıları temsil etme yöntemidir.
- 0'dan 9'a kadar rakamlar ve A'dan F'ye kadar harfler kullanılır.
- İkili (Binary) sayıların (01101011 gibi uzun dizilerin) okunmasını kolaylaştırmak için bir **kısaltma** yöntemi olarak kullanılır.
- Bir **Hexadecimal rakam** (örneğin 'A' veya '5') , tam olarak **4 bit'e** karşılık gelir. İki Hexadecimal rakam ise 8 bit'e(1 bayt) karşılık gelir.
- Örnek:  $(1101\ 1010\ 1100\ 1011)_2 = (DACB)_{16}$

# Metni Bit'lere Çevirmek: ASCII Standardı

En eski ve temel kodlama sistemlerinden biri olan **ASCII**(**American Standard Code for Information Interchange**), İngilizce'deki her bir karakter için 7-bit'lik (genellikle 8-bit'lik bir bayt içinde saklanır) **benzersiz bir desen** tanımlar.



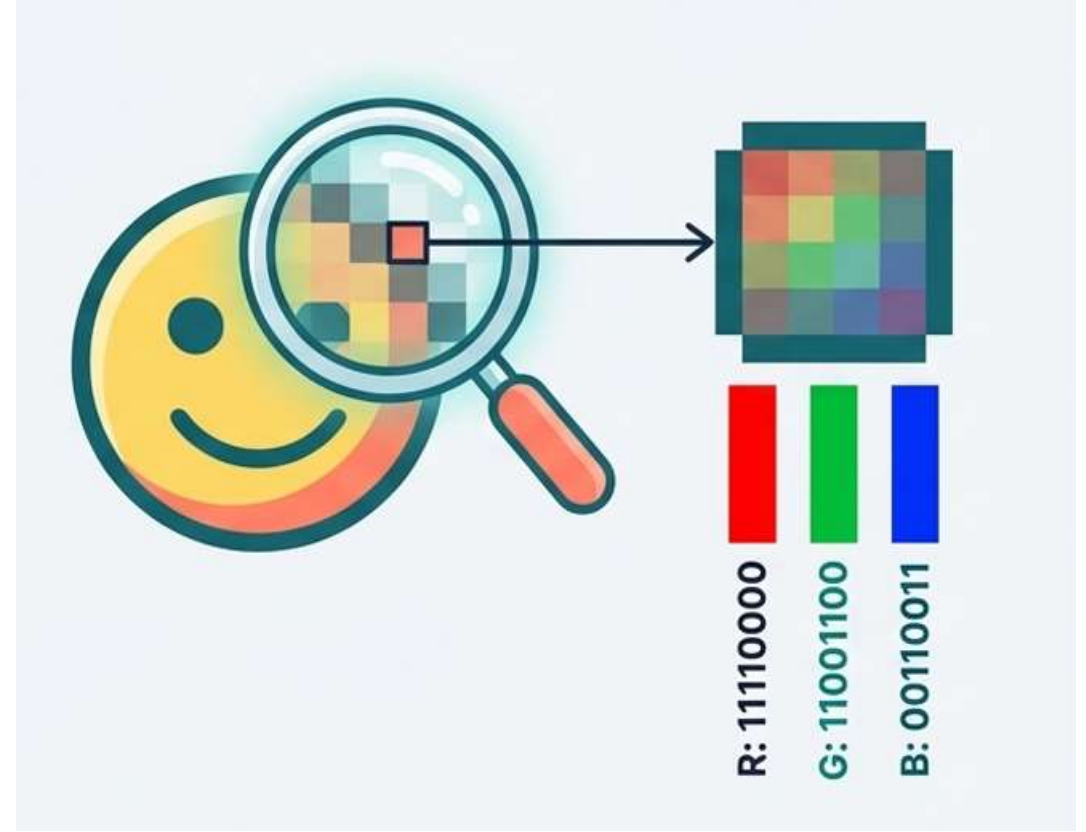
# Metni Bit'lere Çevirmek: Unicode (UTF-8)

- ASCII'nin yetersiz kalması (Türkçe'deki ğ, ş, ı gibi harfleri, Çince, Japonca ve emojileri kodlayamaması) üzerine geliştirilmiştir.
- Tek bir bayttan (8 bit) başlayıp dört bayta (32 bit) kadar uzanabilir. Bu esnek yapı, dünya üzerindeki hemen hemen tüm yazı sistemlerini temsil etme olanağı sunar.
- **UTF-8:** İnternette en yaygın kullanılan Unicode biçimidir. İngiliz alfabesindeki karakterleri **1 bayt** ile, diğer dillerdeki özel karakterleri (örneğin Türkçe karakterler) **2 veya 3 bayt** ile kodlar. Bu sayede bellekten tasarruf sağlar.

# Görüntüyü Bit'lere Çevirmek:Piksellerin Dili

Dijital görüntüler 'bitmap' tekniği ile saklanır.Bir görüntü, 'piksel' adı verilen binlerce küçük renkli noktadan oluşan bir ızgaradır.Her pikselin rengi,tipik olarak Kırmızı,Yeşil,Mavi (RGB) bileşenlerinin yoğunluğunu belirten bir bit deseni ile kodlanır.

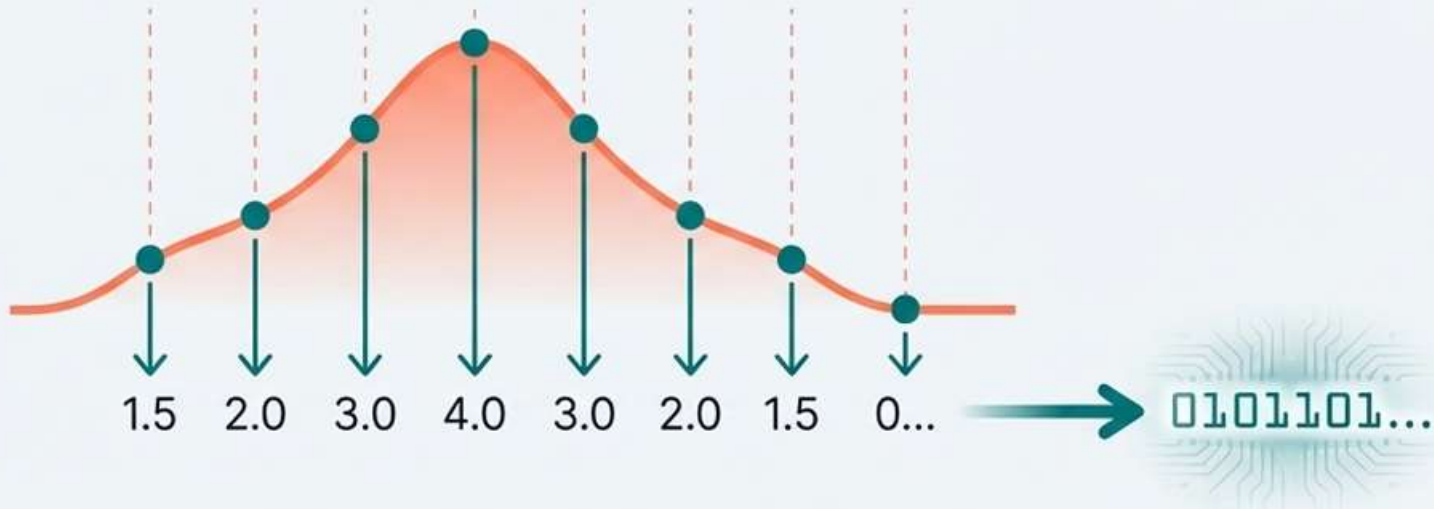
- **Çözünürlük:**Bir görüntüden piksel sayısı ne kadar fazlaysa,görüntü o kadar detaylı olur.
- **Renk Derinliği:**Her piksel için ne kadar çok bit kullanılırsa,o kadar çok renk tonu temsil edilebilir(örneğin,24-bit renk (yaklaşık 16.7 milyon) renk)





# Sesi Bit'lere Çevirmek:Dalgaları Yakalamak

Ses,doğası gereği analog bir dalgadır.Bunu dijitalleştirmek için 'örnekleme' (sampling) tekniği kullanılır.Ses dalgasının genliği (yüksekliği) saniyede binlerce kez ölçülür ve her ölçüm bir sayı olarak kaydedilir.Bu sayılar daha sonra bit desenlerine dönüştürülür.



**Örnekleme Hızı:** Saniyede ne kadar çok örnek alınırsa, sesin dijital kopyası orijinaline o kadar sadık olur.

**CD Kalitesi:** Saniyede 44,100 örnek.

**Telefon Görüşmesi:** Saniyede 8,000 örnek.

## 2.BÖLÜM:

### Sorun: Kaçınılmaz Veri Tufanı

Dünyamızı dijitalleştirmek inanılmaz olanaklar sunar, ancak bir bedeli vardır: devasa miktarda veri.



Bir dakikalık yüksek kaliteli ses, milyonlarca bayt yer kaplayabilir.

Yüksek çözünürlüklü tek bir fotoğraf, megabaytlarca alan gerektirebilir.

Bir film, gigabaytlarca veri anlamına gelir.

**Bu sürekli büyüyen veri yığınına daha verimli bir şekilde nasıl saklayabilir, işleyebilir ve iletebiliriz?**

# Çözüm: Veri Sıkıştırma Sanatı

- **Veri sıkıştırma (Data Compression)**, bir bilginin (veri dosyasının) daha az yer kaplayacak şekilde kodlanması veya temsil edilmesi işlemidir.
- Sıkıştırma algoritmaları, veriyi açarken orijinaline ne kadar sadık kaldıklarına göre iki ana kategoriye ayrılır:
  - ◆ **Kayıpsız Sıkıştırma (Lossless Compression)**
  - ◆ **Kayıplı Sıkıştırma (Lossy Compression)**



# Kayıpsız Sıkıştırma(Lossless)

- Veri sıkıştırılır ve açıldığında **tamamen orijinaliyle aynı** olur. Bilgi kaybı **olmaz**.
- Belirli veri sıkıştırma algoritmaları kullanarak var olan özgün veriyi sıkıştırılmış veri yapar.
- **Kullanım Alanları:** Metin dosyaları, yazılım kodları, veritabanları, bilimsel veriler, tıbbi görüntüler (verinin bütünlüğü kritik olduğu her yer).
- Popüler Algoritmalar:
  - ➡ Run-Length Encoding (RLE)
  - ➡ Huffman Kodlaması
  - ➡ Lempel-Ziv-Welch...

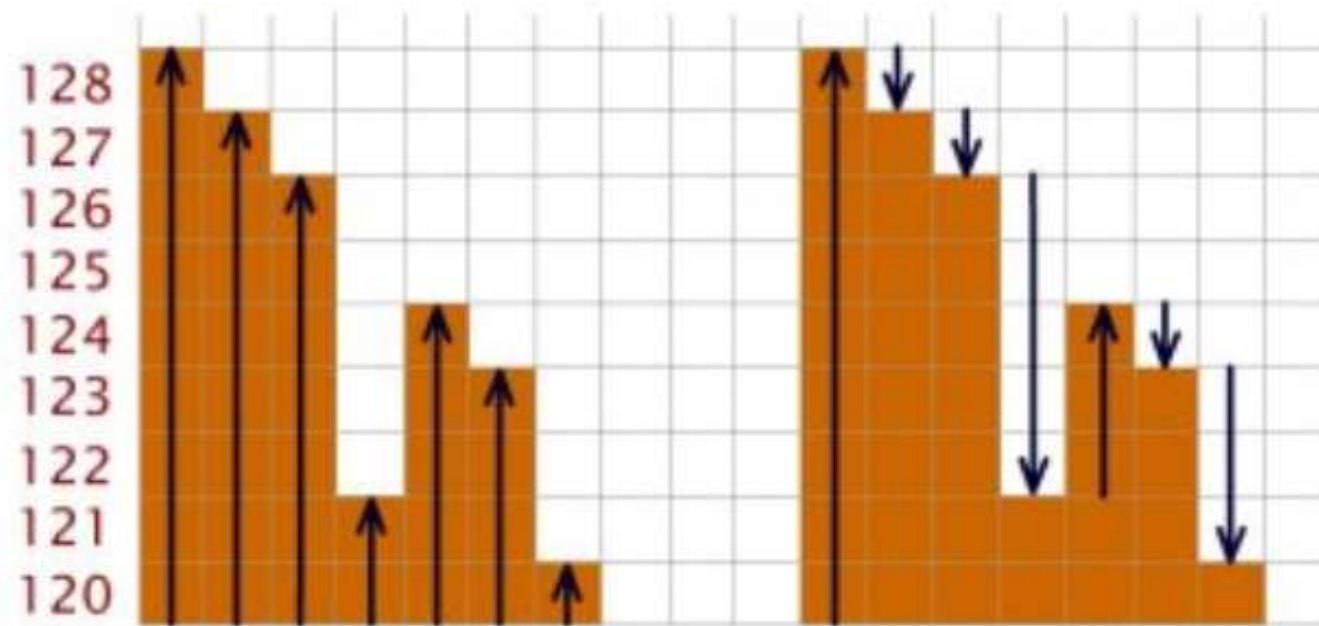
## Numerical example

An example of seven gray pixels:

128, 127, 126, 121, 124, 123, 120

Can be re-written in shorter numbers requiring less bits like:

128, -1, -1, -5, +3, -1, -3



Lossless method

# Kayıplı Sıkıştırma(Lossy)

- Veri sıkıştırılırken, insan algısı için **önemsiz kabul edilen bazı bilgiler kalıcı olarak atılır.** Açılan veri, orijinaline çok yakındır ancak **aynısı değildir.**
- **Kullanım Alanları:** Multimedya dosyaları (görüntü, ses, video) – çünkü insan gözü ve kulağı küçük kayıpları fark etmez.
- Popüler Algoritmalar:
  - ➡ JPEG (Görüntü)
  - ➡ MP3 (Ses)
  - ➡ MPEG (Video)



## Numerical example

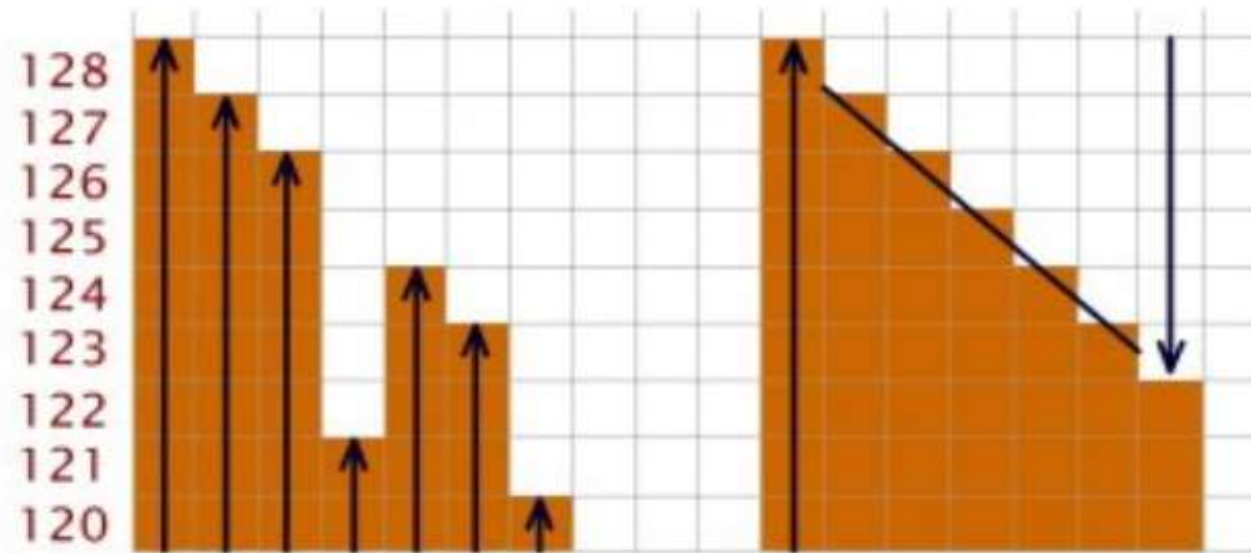
The previous sequence of numbers  
128, 127, 126, 121, 124, 123, 120

can be re-written like:

128 - 6

Result after decompression:

128, 127, 126, 125, 124, 123, 122



Lossy method

## 3.bölüm:

# Zekice Bir Fikir:Run-Length Encoding(RLE)

- RLE, veri sıkıştırma algoritmaları arasında **en basit ve en hızlı** olanlarından biridir.
- RLE, bir veri dizisi (metin, piksel, bayt) içerisindeki **ardışık olarak tekrar eden aynı değerlerin (koşuların)** sayısını ve o değeri kaydederek veriyi kısaltır.
- Verideki **mekansal veya zamansal tekrarı** (redundancy) ortadan kaldırmaktır. Bir değer ne kadar uzun süre ardışık olarak tekrar ediyorsa, RLE o kadar etkili olur.
- **Kayıpsız Yöntem:** Sıkıştırılan veriden orijinal veri, hiçbir bilgi kaybı olmadan **tamamen** geri elde edilebilir.



# RLE Algoritmasının Çalışma Prensipleri:

- Algoritma, veriyi baştan sona tarar ve aynı değeri takip eden kaç tane aynı değerin geldiğini sayar.
1. **Koşu Tespiti:** Ardışık aynı değerlerin başladığı noktayı belirler.
  2. **Koşu Uzunluğu (Length) Sayımı:** Ardışık tekrar eden elemanların sayısını (koşu uzunluğunu) sayar.
  3. **Kodlama:** Ardışık diziyi, genellikle **(Tekrar Sayısı, Değer)** formatında bir çift ile değiştirir.
  4. **Devam Etme:** Sayma işlemi bittikten sonra, bir sonraki farklı değere geçer ve bu süreci tekrar eder.

# Örnek Kodlama Yapısı



# RLE Adım Adım: Kodlama (Encode)

1. Dizinin başından başla. Mevcut karakteri ('A') ve sayacını (1) tut.
2. Bir sonraki karaktere bak. Aynı mı ('A')? Sayacı artır (2, 3, 4, 5).
3. Farklı bir karaktere ('B') mi geldin? Önceki karakterin sayacını ve kendisini (5A) çıktıya yaz.
4. Yeni karakterle ('B') sayacı sıfırla (1) ve 2. adıma geri dön.
5. Dizinin sonuna kadar devam et.



# Sihri Geri Almak :Kod Çözme(Decode)

`decode` fonksiyonu, sıkıştırılmış RLE verisini alıp orijinal haline geri döndürür. Bu, RLE'nin kayıpsız bir algoritma olduğunun kanıtıdır.

1. Sıkıştırılmış diziyi oku: sayı-karakter çiftleri halinde.
2. İlk çifti al: `5` ve `A`.
3. 'A' karakterini 5 kez çıktıya yaz: `AAAAA`.
4. İkinci çifti al: `3` ve `B`.
5. 'B' karakterini 3 kez çıktıya yaz: `AAAAABBB`.
6. Tüm çiftler işlenene kadar devam et.

**Girdi: 5A3B2C1D2A**

5A3B2C1D2A → AAAAA

5A3B2C1D2A → AAAAA BBB

5A3B2C1D2A → AAAAA BBB CC

5A3B2C1D2A → AAAAA BBB CC D

5A3B2C1D2A → AAAAA BBB CC D A A

# Kullanım Alanları:

- RLE'nin verimli olduğu yerler, veri içeriğinin tahmin edilebilir veya tek tip olduğu durumlardır:
  - **Grafik Görüntüleri:** Özellikle **siyah-beyaz** veya **sınırlı renk paletine sahip** görüntülerde (faks, simgeler, diyagramlar). Bu tür görüntülerde uzun, tek renkli alanlar sıkça bulunur.
  - **Tıbbi Görüntüleme:** MRI veya CT taramaları gibi görüntülerde, arka planın genellikle tekdüze olduğu alanlarda etkilidir.
  - **TIFF ve BMP Formatları:** Bu formatların bazı varyantları, kayıpsız sıkıştırma seçeneği olarak RLE'yi kullanır.
  - **Veritabanları:** Ardışık kayıtların aynı değerleri içerdiği durumlarda da uygulanabilir.

# RLE'nin Diğer Algoritmalar İle İlişkisi:

- Basitliği nedeniyle RLE genellikle günümüzün modern formatlarında **tek başına** kullanılmaz, ancak **ikincil bir sıkıştırma adımı** olarak veya başka algoritmaların (Deflate gibi) bir parçası olarak kullanılır.
  - **Deflate/PNG:** PNG görüntü sıkıştırması, veriye RLE benzeri bir filtreden geçirip ardından Huffman kodlaması uygulayarak daha iyi sonuçlar elde eder.



# Başarıyı Ölçmek: Sıkıştırma Oranı

Bir sıkıştırma algoritmasının etkinliği, **sıkıştırma oranı**yla ölçülür. Bu oran, verinin ne kadar küçüldüğünü yüzde olarak ifade eder.

$$\text{Sıkıştırma Oranı (\%)} = (1 - (\text{Sıkıştırılmış Boyut} / \text{Orijinal Boyut})) * 100$$

Orijinal Veri: `AAAAABBBCCDAA` (Boyut: 13 karakter)

Sıkıştırılmış Veri: `5A3B2C1D2A` (Boyut: 8 karakter)

Hesaplama:

$$\text{Oran} = (1 - (8 / 13)) * 100$$

$$\text{Oran} = (1 - 0.615) * 100$$

**Oran  $\approx$  %38.5**

Orijinal Boyut (13)



Sıkıştırılmış Boyut (8)



%38.5 Azalma

# RLE'nin Ötesinde: Sıkıştırma Evreni

Run-Length Encoding, buzdağının sadece görünen kısmı. Veri sıkıştırma, bilgisayar bilimlerinin derin ve aktif bir alanıdır. Günlük hayatta kullandığımız birçok teknoloji, daha karmaşık sıkıştırma algoritmalarına dayanır:



Fotoğraflar için kayıplı sıkıştırma. İnsan gözünün daha az hassas olduğu renk bilgilerini atarak yüksek sıkıştırma oranları elde eder.



Çizimler ve basit animasyonlar için kayıpsız sıkıştırma. Renk paletini sınırlar ve RLE benzeri teknikler kullanır.



Ses için kayıplı sıkıştırma. Psikoakustik modeller kullanarak insanların duyamayacağı verileri ortadan kaldırır.



Video için kayıplı sıkıştırma. Görüntüler arası ve görüntü içi tekrarları ortadan kaldırarak çalışır.





# Az, Daha Çoktur: Sıkıştırmanın Felsefesi

Veri sıkıştırma, sadece bit'leri azaltmak değildir. Bu, dijital dünyada verimlilik, hız ve erişilebilirlik arayışıdır. `AAAAABBBCCDAA`'yı `5A3B2C1D2A`'ya dönüştürmek gibi basit bir fikir, daha hızlı web siteleri, daha küçük e-posta ekleri ve cihazlarımızda daha fazla fazla anı saklama gücü anlamına gelir. Yazacağınız kod, bu temel ama güçlü prensibin bir parçası olacak.